FRONTEND INTERVIEW PREPARATION • 2026 EDITION

JAVASCRIPT INTERVIEW MASTER GUIDE

Learn in the right order. Explain with confidence. Code with clarity.

English interview answers • தமிழ் விளக்கம் • Syntax • Working examples • Real-world use • Practice tasks

JS

For frontend students | Beginner → Interview Ready
Prepared as a classroom-friendly reference guide

START HERE

How to use this book

This guide is designed for students who have finished HTML, CSS, and frontend JavaScript but have not yet studied backend development. The topics follow a dependency-based order: basics → data structures → functions → browser DOM → asynchronous JavaScript → APIs → interview depth.

For each topic: read the two short English paragraphs aloud as an interview answer; read the Tamil explanation to confirm understanding; type the syntax and code without copying; explain one real-world use case; then answer the interview question.

இந்த book-ஐ மனப்பாடம் செய்ய வேண்டாம். Topic-ஐ புரிந்து English-ல் இரண்டு paragraph பேசிப் practice செய்யுங்கள். Code-ஐ நீங்களே type செய்து output predict பண்ணுங்கள். Backend தெரியாத students-க்கும் Fetch API வரை புரியும் மாதிரி order அமைக்கப்பட்டுள்ளது.

The complete learning order

01	**01 FOUNDATION** Start here: language basics	8 topics
02	**02 CORE JS** Build data and function confidence	12 topics
03	**03 FUNCTIONS & RUNTIME** Understand how JavaScript behaves	4 topics
04	**04 BROWSER DOM** Make frontend pages interactive	6 topics
05	**05 ASYNCHRONOUS JS** Connect frontend with APIs	6 topics
06	**06 INTERVIEW DEPTH** Become interview-ready	6 topics

Daily interview routine (60–90 minutes)

15 min — Review yesterday's concepts and predict outputs.
25 min — Explain 2–3 topics aloud in English.
30 min — Solve one logic problem and one DOM/API problem.
10 min — Write mistakes in a revision notebook.

Important: This guide builds readiness; no document can guarantee selection. Interviews also test communication, coding accuracy, projects, and problem solving.

01

FOUNDATION

Start here: language basics. Finish this unit before moving to the next one.

8 INTERVIEW TOPICS

01.01 JavaScript & How It Runs

SAY THIS IN YOUR INTERVIEW • ENGLISH

> JavaScript is a programming language that makes web pages interactive. HTML provides structure, CSS provides design, and JavaScript handles behavior such as form validation, button clicks, and displaying data.
>
> In a browser, JavaScript runs through the JavaScript engine. I use it to read user input, update the DOM, and communicate with APIs without refreshing the entire page.

தமிழில் புரிந்து கொள்ள

JavaScript என்பது website-க்கு action கொடுக்கும் language. Button click, form validation, table data show செய்வது எல்லாம் JS மூலம் செய்யலாம்.

SYNTAX

```
<script src="app.js" defer></script>
```

WORKING EXAMPLE

```
console.log("Welcome");
document.title = "Student Portal";
```

Real-world use: Login forms, dashboards, product listings.

Interview question: What is JavaScript and how is it different from HTML/CSS?

01.02 Variables: var, let, const

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Variables store values. I normally use const when the variable should not be reassigned and let when its value must change. var is older and has function scope.
>
> The main difference is scope and reassignment. let and const are block-scoped; var is function-scoped. A const object can still have its properties changed.

தமிழில் புரிந்து கொள்ள

Value store செய்ய variables use பண்ணுவோம். Reassign ஆகாத variable-க்கு const; value change ஆகணும்னா let. var பழைய method.

SYNTAX

```
let age = 20; const name = "Arun";
```

WORKING EXAMPLE

```
let score = 10;
score = 15;
const student = { name: "Arun" };
student.name = "Priya";
console.log(score, student.name);
```

Real-world use: Form values, API response variables, counters.

Interview question: Difference between var, let, const?

01.03 Data Types & typeof

SAY THIS IN YOUR INTERVIEW • ENGLISH

> JavaScript has primitive values such as string, number, boolean, undefined, null, bigint, and symbol. Objects, arrays, and functions are reference values.
>
> I check types using typeof, but I remember that typeof null is "object" because of a historical behavior. For arrays, Array.isArray() is more accurate.

தமிழில் புரிந்து கொள்ள

String, number, boolean போன்றவை basic data types. Array, object data-வை group செய்ய உதவும். typeof வைத்து type பார்க்கலாம்.

SYNTAX

```
typeof value; Array.isArray(value);
```

WORKING EXAMPLE

```
let name = "Mugund";
let age = 22;
let active = true;
let marks = [70, 80];
console.log(typeof name); // string
console.log(Array.isArray(marks)); // true
```

Real-world use: Checking form inputs and handling API data.

Interview question: Why does typeof null return object?

01.04 Operators & Type Coercion

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Operators perform arithmetic, comparison, logical, and assignment operations. I use === when comparing values because it checks both type and value.

> JavaScript can convert values automatically in some operations. For example, "5" + 2 produces "52", while "5" - 2 produces 3. Understanding coercion prevents bugs.

தமிழில் புரிந்து கொள்ள

=== type-உம் value-உம் check பண்ணும். == சில நேரம் type convert பண்ணும். அதனால் comparison-க்கு === நல்லது.

SYNTAX

```
+, -, *, /, %, ===, !==, &&, ||, ??
```

WORKING EXAMPLE

```
console.log(5 === "5"); // false
console.log(5 == "5");  // true
console.log("5" + 2); // "52"
console.log(10 % 3); // 1
```

Real-world use: Login checks, conditional validation, calculations.

Interview question: Difference between == and ===?

01.05 Truthy, Falsy & Nullish Values

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Conditions do not always need a boolean. JavaScript converts values to truthy or falsy. Common falsy values include false, 0, empty string, null, undefined, and NaN.
>
> I use || for a general fallback and ?? when I only want a fallback for null or undefined. This matters when zero is a valid input.

தமிழில் புரிந்து கொள்ள

Empty string, 0, null போன்றவை falsy. 0 valid value ஆக இருந்தால் || விட ?? சரியாக இருக்கும்.

SYNTAX

```
value || fallback; value ?? fallback;
```

WORKING EXAMPLE

```
let quantity = 0;
console.log(quantity || 10); // 10
console.log(quantity ?? 10); // 0
console.log(Boolean("")); // false
```

Real-world use: Default quantities, optional profile fields, UI fallbacks.

Interview question: Difference between || and ??

01.06 Conditions: if, else, switch

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Conditional statements let my program make decisions. I use if/else for ranges and complex conditions, and switch when I compare one value against several exact cases.
>
> In frontend applications, conditions control error messages, permissions, loading indicators, and which content is shown to the user.

தமிழில் புரிந்து கொள்ள

Condition true என்றால் ஒரு action, false என்றால் வேறொரு action. Student pass/fail, login status போன்றவற்றில் use ஆகும்.

SYNTAX

```
if (condition) { ... } else { ... }
```

WORKING EXAMPLE

```
let mark = 68;
if (mark >= 35) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

Real-world use: Form validation and conditional rendering.

Interview question: When would you use switch instead of if?

01.07 Loops: for, while, do...while

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Loops repeat code. I use for when I know the iteration count, while when repetition depends on a condition, and do...while when the body must run at least once.
>
> Loops are useful for calculating totals, searching records, and printing data. I also know break stops a loop and continue skips one iteration.

தமிழில் புரிந்து கொள்ள

ஒரே code-ஐ பலமுறை run செய்ய loop. for count தெரிந்தால் use பண்ணலாம்; while condition அடிப்படையில் run ஆகும்.

SYNTAX

```
for (let i=0; i<5; i++) { ... }
```

WORKING EXAMPLE

```
let total = 0;
for (let i = 1; i <= 5; i++) {
  total += i;
}
console.log(total); // 15
```

Real-world use: Invoice totals, iterating student marks, table rows.

Interview question: Difference between for, while, do...while?

01.08 Functions & Return

SAY THIS IN YOUR INTERVIEW • ENGLISH

> A function is a reusable block of code. Parameters receive inputs, and return sends a result back to the caller. I use functions to avoid repeating logic.
>
> A function can be declared normally or stored as an expression. When a function does not explicitly return a value, its result is undefined.

தமிழில் புரிந்து கொள்ள

ஒரே logic-ஐ பல இடத்தில் use செய்ய function. Parameter input வாங்கும்; return output கொடுக்கும்.

SYNTAX

```
function add(a,b) { return a+b; }
```

WORKING EXAMPLE

```
function add(a, b) {
  return a + b;
}
const result = add(10, 20);
console.log(result); // 30
```

Real-world use: Reusable validation and salary calculations.

Interview question: Difference between console.log and return?

02

CORE JS

Build data and function confidence. Finish this unit before moving to the next one.

12 INTERVIEW TOPICS

02.01 Arrow Functions

SAY THIS IN YOUR INTERVIEW • ENGLISH

> An arrow function is a shorter way to write a function expression. I use arrow functions often in callbacks and array methods because they make code compact.
>
> Arrow functions do not create their own this or arguments binding. For object methods that depend on their own this, a normal method is often a better choice.

தமிழில் புரிந்து கொள்ள

Arrow function என்பது function எழுதும் short syntax. forEach, map, event callback போன்ற இடங்களில் அதிகம் வரும்.

SYNTAX

```
const add = (a,b) => a+b;
```

WORKING EXAMPLE

```
const square = n => n * n;
console.log(square(5)); // 25
const greet = name => `Hi ${name}`;
console.log(greet("Arun"));
```

Real-world use: Array callbacks, event handlers, small utilities.

Interview question: Arrow function vs normal function?

02.02 Scope & Lexical Scope

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Scope decides where a variable can be accessed. JavaScript has global, function, and block scope. let and const stay inside the block where they are declared.
>
> Lexical scope means a function can access variables based on where it was written, not where it was called. This idea is the foundation of closures.

தமிழில் புரிந்து கொள்ள

Variable எங்கே declare பண்ணுறோமோ அதைப் பொறுத்து access மாறும். Outer variable-ஐ inner function access செய்யலாம்.

SYNTAX

```
{ let x = 1; } // x outside is unavailable
```

WORKING EXAMPLE

```
const city = "Chennai";
function show() {
  let area = "Anna Nagar";
  console.log(city, area);
}
show();
```

Real-world use: Avoiding variable collisions and protecting local values.

Interview question: What is block scope?

02.03 Hoisting & Temporal Dead Zone

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Hoisting describes how declarations are processed before execution. Function declarations can be called before their position in the source code.
>
> var is initialized to undefined during setup. let and const exist in their scope before declaration but cannot be accessed until initialized; that period is called the temporal dead zone.

தமிழில் புரிந்து கொள்ள

Function declaration மேலே call செய்தாலும் வேலை செய்யலாம். let/const declare பண்ணுவதற்கு முன் access செய்தால் error வரும்.

SYNTAX

```
sayHi(); function sayHi() { ... }
```

WORKING EXAMPLE

```
sayHi();
function sayHi() {
  console.log("Hi");
}
// console.log(x); // ReferenceError
let x = 10;
```

Real-world use: Debugging initialization and ReferenceError problems.

Interview question: What is hoisting? Explain TDZ.

02.04 Strings & Template Literals

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Strings store text. I use methods such as trim, includes, slice, and toLowerCase for common input handling. Template literals use backticks and support ${} expressions.
>
> Template literals make it easy to combine variables with HTML or messages. They also support multiline strings, which is useful when rendering simple DOM content.

தமிழில் புரிந்து கொள்ள

Backtick (`) use பண்ணி ${} மூலம் value-ஐ string-க்குள் insert செய்யலாம். trim() space remove செய்யும்.

SYNTAX

```
`Hello ${name}`; text.trim();
```

WORKING EXAMPLE

```
const name = "Priya";
const course = "JavaScript";
console.log(`${name} learns ${course}`);
console.log("  hi  ".trim());
```

Real-world use: Dynamic greeting messages and HTML table rows.

Interview question: What is a template literal?

02.05 Arrays & Basic Methods

SAY THIS IN YOUR INTERVIEW • ENGLISH

> An array stores multiple values in order. Each element has a zero-based index. I use push to add at the end, pop to remove from the end, and length to count items.
>
> Arrays are useful for holding student lists, cart items, and API results. I check the array contents before deciding how to display or update the data.

தமிழில் புரிந்து கொள்ள

Array-ல் multiple values store பண்ணலாம். Index 0-ல start ஆகும். push add; pop remove.

SYNTAX

```
const items = []; items.push(value);
```

WORKING EXAMPLE

```
const marks = [50, 70];
marks.push(90);
console.log(marks[0]); // 50
console.log(marks.length); // 3
marks.pop();
```

Real-world use: Shopping cart and list management.

Interview question: Difference between push/pop and shift/unshift?

02.06 Objects & Nested Objects

SAY THIS IN YOUR INTERVIEW • ENGLISH

> An object groups related data as key-value pairs. I access values using dot notation or bracket notation. Objects are useful for representing one student, user, or product.
>
> Real API responses often contain nested objects. For example, a user can have an address object, so I access user.address.city to read the city.

தமிழில் புரிந்து கொள்ள

Object-ல் ஒரு student-க்கு name, age, course போன்ற properties இருக்கும். Nested object-க்கும் dot notation use பண்ணலாம்.

SYNTAX

```
obj.name; obj["name"]; obj.address.city;
```

WORKING EXAMPLE

```
const user = {
  name: "Arun",
  address: { city: "Chennai" }
};
console.log(user.name);
console.log(user.address.city);
```

Real-world use: User profile, order details, API response handling.

Interview question: Array vs object?

02.07 Array of Objects

SAY THIS IN YOUR INTERVIEW • ENGLISH

> An array of objects stores multiple structured records. Each object represents one item, and the array represents the collection.
>
> I commonly receive arrays of objects from APIs. I can loop through them to build student tables, employee cards, and product lists.

தமிழில் புரிந்து கொள்ள

Students list-ல் ஒவ்வொரு student-உம் object; எல்லா students-உம் array. Frontend API data-வில் இது ரொம்ப common.

SYNTAX

```
const users = [{id:1,name:"A"}, ...];
```

WORKING EXAMPLE

```
const students = [
  { id: 1, name: "Arun" },
  { id: 2, name: "Priya" }
];
students.forEach(s => console.log(s.name));
```

Real-world use: Data tables, search results, employee directories.

Interview question: How do you access a property inside an array of objects?

02.08 forEach, map, filter & find

SAY THIS IN YOUR INTERVIEW • ENGLISH

> forEach runs a callback for every item, mainly for side effects such as logging or building output. map creates a new array by transforming every element.
>
> filter creates a new array of matching items, while find returns the first matching element or undefined. I choose based on whether I need all items, a new transformed array, or one record.

தமிழில் புரிந்து கொள்ள

forEach loop செய்யும்; map new array உருவாக்கும்; filter matching values மட்டும் தரும்; find first matching item தரும்.

SYNTAX

```
arr.forEach(fn); arr.map(fn); arr.filter(fn); arr.find(fn);
```

WORKING EXAMPLE

```
const nums = [10, 20, 30];
nums.forEach(n => console.log(n));
console.log(nums.map(n => n + 5));
console.log(nums.filter(n => n > 15));
console.log(nums.find(n => n > 15));
```

Real-world use: Render lists, update salaries, filter products, find user.

Interview question: map vs forEach; find vs filter?

02.09 some, every, reduce, sort

SAY THIS IN YOUR INTERVIEW • ENGLISH

> some checks whether at least one element matches a condition, while every checks whether all elements match. reduce combines array items into one result.
>
> sort sorts array elements in place, and numeric sorting needs a comparison function. I use these methods for validations, totals, and rankings.

தமிழில் புரிந்து கொள்ள

some = ஒருத்தராவது match; every = எல்லாரும் match; reduce = total; sort = order.

SYNTAX

```
arr.some(fn); arr.every(fn); arr.reduce(fn,0);
```

WORKING EXAMPLE

```
const marks = [50, 70, 90];
console.log(marks.every(m => m >= 35));
console.log(marks.some(m => m > 80));
console.log(marks.reduce((a,b) => a+b, 0));
console.log([...marks].sort((a,b) => b-a));
```

Real-world use: Cart totals, eligibility checks, ranking scores.

Interview question: Why is sort((a,b)=>a-b) needed for numbers?

02.10 Destructuring, Spread & Rest

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Destructuring extracts values from arrays or objects into variables. Spread expands values into a new array or object. Rest collects multiple values into one parameter.
>
> I use spread to make shallow copies before updating arrays and objects, especially in frontend state management. A shallow copy does not deeply clone nested objects.

தமிழில் புரிந்து கொள்ள

Destructuring value எடுக்க; spread copy/merge செய்ய; rest பல arguments collect செய்ய.

SYNTAX

```
const {name}=user; const copy={...user};
```

WORKING EXAMPLE

```
const user = { name: "Arun", age: 22 };
const { name } = user;
const updated = { ...user, age: 23 };
const nums = [1, 2];
const next = [...nums, 3];
console.log(name, updated, next);
```

Real-world use: Updating form objects, React state, merging API data.

Interview question: Spread vs rest? Shallow vs deep copy?

02.11 Optional Chaining & Nullish Coalescing

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Optional chaining safely accesses a property that may not exist. Instead of throwing an error when an intermediate value is null or undefined, it returns undefined.
>
> Nullish coalescing provides a fallback only when a value is null or undefined. I use both when API fields are optional or loading is incomplete.

தமிழில் புரிந்து கொள்ள

API-ல் address இல்லாமலும் இருக்கலாம். user?.address?.city error avoid செய்யும். ?? default value தரும்.

SYNTAX

```
user?.address?.city ?? "Unknown"
```

WORKING EXAMPLE

```
const user = { name: "Arun" };
const city = user?.address?.city ?? "Unknown";
console.log(city); // Unknown
```

Real-world use: Optional API fields and incomplete profiles.

Interview question: What is optional chaining?

02.12 Equality, References & Copying

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Primitive values are compared by their values, while objects and arrays are compared by reference. Two different objects with identical properties are not strictly equal.
>
> Assigning an object to another variable shares the same reference. To avoid accidentally changing the original top-level properties, I can create a shallow copy using spread.

தமிழில் புரிந்து கொள்ள

Object copy செய்யாமல் assign பண்ணினா இரண்டும் same reference. ஒரு object change பண்ணினால் மற்றதிலும் reflect ஆகும்.

SYNTAX

```
const copy = {...original};
```

WORKING EXAMPLE

```
const a = { score: 10 };
const b = a;
b.score = 20;
console.log(a.score); // 20
console.log({x:1} === {x:1}); // false
```

Real-world use: Preventing accidental data mutation in UI.

Interview question: Why is [] === [] false?

03

FUNCTIONS & RUNTIME

Understand how JavaScript behaves. Finish this unit before moving to the next one.

4 INTERVIEW TOPICS

03.01 Closures

SAY THIS IN YOUR INTERVIEW • ENGLISH

> A closure happens when an inner function remembers variables from its outer lexical scope, even after the outer function has finished running.
>
> I use closures when I want to preserve a private counter or configuration. Storing the returned function lets me call the same closure repeatedly without resetting its remembered variables.

தமிழில் புரிந்து கொள்ள

> Outer function முடிந்த பிறகும் inner function outer variable-ஐ நினைவில் வைத்திருக்கும். Returned function-ஐ variable-ல் store செய்தால் same count continue ஆகும்.

SYNTAX

```
const saved = outer(); saved();
```

WORKING EXAMPLE

```
const counter = () => {
  let count = 0;
  return () => ++count;
};
const next = counter();
console.log(next()); // 1
console.log(next()); // 2
console.log(next()); // 3
```

Real-world use: Private counters, event handler factories, configuration.

Interview question: Why does counter()() reset while saved() continues?

03.02 Callbacks & Higher-Order Functions

SAY THIS IN YOUR INTERVIEW • ENGLISH

> A callback is a function passed to another function to be executed when needed. A higher-order function receives a function or returns a function.
>
> Callbacks can be synchronous, such as forEach, or asynchronous, such as a timer callback. Passing the function reference is different from calling it immediately.

தமிழில் புரிந்து கொள்ள

Function-ஐ argument-ஆ pass பண்ணுவது callback. forEach synchronous; setTimeout callback later run ஆகும்.

SYNTAX

```
run(callback); arr.forEach(item => ...);
```

WORKING EXAMPLE

```
const calculate = (a, b, action) => {
  return action(a, b);
};
const add = (a, b) => a + b;
console.log(calculate(10, 5, add)); // 15
```

Real-world use: Event handlers, array processing, timers.

Interview question: What is a callback? Is every callback async?

03.03 this, call, apply & bind

SAY THIS IN YOUR INTERVIEW • ENGLISH

> The value of this depends on how a regular function is called. In an object method, this commonly refers to the object. Arrow functions capture this lexically.
>
> call and apply invoke a function with a chosen this value, while bind returns a new function with that this value attached. I understand these because event handlers and object methods can behave differently.

தமிழில் புரிந்து கொள்ள

Regular function-ல் this call செய்யும் context-ஐ பொறுத்து மாறும். Arrow function தனி this create பண்ணாது.

SYNTAX

```
fn.call(obj, a); fn.apply(obj,[a]); fn.bind(obj);
```

WORKING EXAMPLE

```
const person = {
  name: "Arun",
  show() { console.log(this.name); }
};
person.show();
const other = { name: "Priya" };
person.show.call(other);
```

Real-world use: Object methods, event callbacks, reusable functions.

Interview question: How is this different in arrow functions?

03.04 Execution Context, Call Stack & Event Loop

SAY THIS IN YOUR INTERVIEW • ENGLISH

> JavaScript executes synchronous code using a call stack. When a function is called, its execution is added to the stack; it is removed after completion.
>
> The event loop coordinates queued callbacks with the call stack. Promise reactions run as microtasks, generally before timer callbacks queued as tasks, once the current synchronous code finishes.

தமிழில் புரிந்து கொள்ள

JS synchronous code முதலில் run ஆகும். Promise .then microtask; setTimeout task. Stack காலியான பிறகு queued work execute ஆகும்.

SYNTAX

```
console.log(); Promise.resolve().then(); setTimeout();
```

WORKING EXAMPLE

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
// A D C B
```

Real-world use: Understanding UI delays and async output order.

Interview question: Predict output: sync, Promise, setTimeout.

04

BROWSER DOM

Make frontend pages interactive. Finish this unit before moving to the next one.

6 INTERVIEW TOPICS

04.01 DOM & Element Selection

SAY THIS IN YOUR INTERVIEW • ENGLISH

> The DOM is the browser representation of an HTML document as objects and nodes. JavaScript uses it to read and change page content.
>
> I select elements using getElementById or querySelector, then update properties such as textContent, value, and innerHTML. I use textContent for plain user-provided text.

தமிழில் புரிந்து கொள்ள

HTML browser-ல் DOM tree ஆக மாறும். JS மூலம் element select செய்து content/style change செய்யலாம்.

SYNTAX

```
document.getElementById("id"); document.querySelector(".box");
```

WORKING EXAMPLE

```
// HTML: <h2 id="title">Old</h2>
const title = document.getElementById("title");
title.textContent = "New Title";
```

Real-world use: Dynamic headings, status messages, form updates.

Interview question: Difference between innerHTML and textContent?

04.02 Events & addEventListener

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Events represent user interactions such as click, input, change, and submit. I use addEventListener to connect a function to an event.
>
> The event object contains details about the interaction. For forms, I call preventDefault to stop the browser from performing its normal submit navigation.

தமிழில் புரிந்து கொள்ள

Button click, typing, submit எல்லாம் event. addEventListener மூலம் function execute செய்யலாம்.

SYNTAX

```
element.addEventListener("click", handler);
```

WORKING EXAMPLE

```
// HTML: <button id="btn">Click</button>
document.getElementById("btn")
  .addEventListener("click", () => {
    console.log("Clicked");
  });
```

Real-world use: Buttons, dropdowns, validation, search inputs.

Interview question: onclick vs addEventListener?

04.03 Form Handling & Validation

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Form handling means reading input values, validating them, and responding to submission. I normally listen to the submit event and prevent default navigation when handling the form in JavaScript.
>
> I trim text inputs, convert numeric inputs when necessary, and show a clear message if data is invalid. After success, I can add a record to an array and re-render the table.

தமிழில் புரிந்து கொள்ள

Form submit ஆகும்போது values எடுத்து check பண்ணி array-க்கு push செய்து table-ல் show செய்யலாம்.

SYNTAX

```
form.addEventListener("submit", e => {e.preventDefault();});
```

WORKING EXAMPLE

```
// HTML: <form id="f"><input id="n" required><button>Save</button></form>
document.getElementById("f").addEventListener("submit", e => {
  e.preventDefault();
  const name = document.getElementById("n").value.trim();
  if (!name) return;
  console.log(name);
});
```

Real-world use: Student registration, login and contact forms.

Interview question: Why do we use preventDefault()?

04.04 Render Array of Objects with forEach

SAY THIS IN YOUR INTERVIEW • ENGLISH

> I can build HTML rows from an array of objects using forEach and template literals. Each iteration adds one row to a string, and I set innerHTML once after the loop.

> This is a simple approach for beginner frontend projects. For production, I avoid putting untrusted values directly into innerHTML without sanitization because it can introduce cross-site scripting risks.

தமிழில் புரிந்து கொள்ள

Array of objects-ஐ forEach loop செய்து backtick HTML rows உருவாக்கலாம். கடைசியில் innerHTML set பண்ணலாம்.

SYNTAX

```
rows += `<tr><td>${item.name}</td></tr>`;
```

WORKING EXAMPLE

```
const users = [{name:"Arun"},{name:"Priya"}];
let rows = "";
users.forEach(u => {
  rows += `<tr><td>${u.name}</td></tr>`;
});
// HTML: <table><tbody id="body"></tbody></table>
document.getElementById("body").innerHTML = rows;
```

Real-world use: Student tables and simple admin lists.

Interview question: Why build the string before setting innerHTML?

04.05 Event Bubbling & Delegation

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Events commonly bubble from a child element up through its ancestors. Event delegation uses a listener on a parent to handle events from its children.
>
> Delegation is useful when list items are created dynamically. Instead of attaching a listener to every new button, I can listen on the table container and check the clicked target.

தமிழில் புரிந்து கொள்ள

Child-ல் click செய்த event parent-க்கும் bubble ஆகலாம். Dynamic buttons-க்கு parent listener useful.

SYNTAX

```
parent.addEventListener("click", e => {...});
```

WORKING EXAMPLE

```
// HTML: <div id="list"><button data-id="7">Open</button></div>
document.getElementById("list").addEventListener("click", e => {
  if (e.target.matches("button[data-id]")) {
    console.log(e.target.dataset.id);
  }
});
```

Real-world use: Dynamic delete/edit buttons in tables.

Interview question: What is event delegation?

04.06 LocalStorage & JSON

SAY THIS IN YOUR INTERVIEW • ENGLISH

> localStorage stores string data in the browser and usually keeps it across page reloads. I use JSON.stringify to save arrays or objects and JSON.parse to read them back.
>
> It is useful for simple preferences or practice apps, but I do not store passwords or sensitive tokens in localStorage. It is not a replacement for a secure backend database.

தமிழில் புரிந்து கொள்ள

localStorage browser-ல் string store செய்யும். Array/object save செய்ய JSON.stringify, read செய்ய JSON.parse.

SYNTAX

```
localStorage.setItem(key, JSON.stringify(data));
```

WORKING EXAMPLE

```
const students = [{name:"Arun"}];
localStorage.setItem("students", JSON.stringify(students));
const saved = JSON.parse(localStorage.getItem("students") || "[]");
console.log(saved);
```

Real-world use: Theme preferences and practice todo lists.

Interview question: Why stringify before localStorage?

05

ASYNCHRONOUS JS

Connect frontend with APIs. Finish this unit before moving to the next one.

6 INTERVIEW TOPICS

05.01 Synchronous vs Asynchronous

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Synchronous statements execute in order on the current call stack. Asynchronous APIs allow operations such as timers and network requests to finish later without blocking the whole page while waiting.
>
> I use asynchronous code for fetching data and waiting for events. I must understand that starting a request does not mean the response is already available on the next line.

தமிழில் புரிந்து கொள்ள

Sync code order-ஆ execute ஆகும். API request போன்ற வேலை result later தரும்; அதற்காக async handle செய்ய வேண்டும்.

SYNTAX

```
setTimeout(callback, 1000);
```

WORKING EXAMPLE

```
console.log("Start");
setTimeout(() => console.log("Later"), 1000);
console.log("End");
// Start, End, Later
```

Real-world use: Network calls and delayed UI messages.

Interview question: Is JavaScript synchronous or asynchronous?

05.02 Promises: resolve, reject, then, catch

SAY THIS IN YOUR INTERVIEW • ENGLISH

> A Promise represents the eventual success or failure of an operation. It begins pending and later settles as fulfilled or rejected.
>
> I use then to handle a fulfilled value and catch to handle a rejection. A function must return its Promise if callers need to wait for its result.

தமிழில் புரிந்து கொள்ள

Promise future result represent பண்ணும். resolve success; reject failure; then success handle; catch error handle.

SYNTAX

```
new Promise((resolve,reject)=>{...});
```

WORKING EXAMPLE

```
const check = age => new Promise((resolve, reject) => {
  age >= 18 ? resolve("Eligible") : reject("Not eligible");
});
check(20).then(console.log).catch(console.error);
```

Real-world use: API results, delayed validation, async workflows.

Interview question: What are the three Promise states?

05.03 Async/Await & try/catch

SAY THIS IN YOUR INTERVIEW • ENGLISH

> An async function returns a Promise. Inside it, await pauses that function until the awaited Promise settles, while the rest of the application can continue.
>
> I wrap awaited operations in try/catch to handle rejected Promises. Async/await usually makes multi-step asynchronous logic easier to read than deeply chained callbacks.

தமிழில் புரிந்து கொள்ள

async function-க்குள் await Promise result வரும்வரை wait செய்யும். Error catch செய்ய try/catch use பண்ணலாம்.

SYNTAX

```
const load = async () => {try {await task()} catch(e) {}};
```

WORKING EXAMPLE

```
const task = () => Promise.resolve("Done");
const run = async () => {
  try {
    const value = await task();
    console.log(value);
  } catch (error) {
    console.log(error.message);
  }
};
run();
```

Real-world use: Fetching users and sequential UI operations.

Interview question: Does await block the entire browser?

05.04 Fetch API: GET Request

SAY THIS IN YOUR INTERVIEW • ENGLISH

> fetch sends an HTTP request and returns a Promise for a Response object. I check response.ok because an HTTP error status such as 404 does not automatically reject the fetch Promise.
>
> After receiving a successful response, I call response.json to parse the JSON body. That method is asynchronous too, so I await it before using the data.

தமிழில் புரிந்து கொள்ள

fetch API-க்கு request அனுப்பும். response.ok check பண்ணி response.json() மூலம் data எடுக்க வேண்டும்.

SYNTAX

```
const response = await fetch(url); const data = await response.json();
```

WORKING EXAMPLE

```
const loadUsers = async () => {
  try {
    const res = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!res.ok) throw new Error(`HTTP ${res.status}`);
    const users = await res.json();
    users.forEach(u => console.log(u.name));
  } catch (err) { console.log(err.message); }
};
loadUsers();
```

Real-world use: Displaying backend users, products and posts.

Interview question: Why check response.ok after fetch?

05.05 Fetch API: POST, PUT, PATCH, DELETE

SAY THIS IN YOUR INTERVIEW • ENGLISH

> HTTP methods communicate the intention of a request. GET reads data, POST commonly creates, PUT replaces a resource, PATCH partially updates, and DELETE requests removal.
>
> In fetch, I set method, headers, and body when needed. JSON requests typically use Content-Type: application/json and JSON.stringify for the request body.

தமிழில் புரிந்து கொள்ள

GET data fetch; POST add; PUT replace; PATCH partial update; DELETE remove. Backend இல்லாமலும் public test API மூலம் practice செய்யலாம்.

SYNTAX

```
fetch(url,{method:"POST",headers:{...},body:JSON.stringify(data)})
```

WORKING EXAMPLE

```
const createPost = async () => {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title: "Hello", body: "Test", userId: 1 })
  });
  if (!res.ok) throw new Error("Request failed");
  console.log(await res.json());
};
createPost().catch(console.error);
```

Real-world use: Create/edit/delete forms; demo API simulates writes.

Interview question: Difference between PUT and PATCH?

05.06 Loading, Error & Empty States

SAY THIS IN YOUR INTERVIEW • ENGLISH

> A reliable frontend does not only show successful data. It also shows loading while the request is in progress, an error when it fails, and an empty state when no records are returned.
>
> I disable duplicate actions during loading and always restore the UI in finally. This gives users feedback and avoids confusing blank screens.

தமிழில் புரிந்து கொள்ள

API load ஆகும்போது Loading, fail ஆனால் Error, data இல்லையென்றால் No records என்று show செய்ய வேண்டும்.

SYNTAX

```
try {...} catch(e) {...} finally {...}
```

WORKING EXAMPLE

```
const load = async () => {
  console.log("Loading...");
  try {
    const r = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!r.ok) throw new Error("Request failed");
    const users = await r.json();
    console.log(users.length ? users.length : "No users");
  } catch (e) { console.log(e.message); }
  finally { console.log("Loading finished"); }
};
load();
```

Real-world use: Production-like dashboards and search pages.

Interview question: How do you handle failed API requests?

06

INTERVIEW DEPTH

Become interview-ready. Finish this unit before moving to the next one.

6 INTERVIEW TOPICS

06.01 Error Handling & Debugging

SAY THIS IN YOUR INTERVIEW • ENGLISH

> JavaScript errors can happen at syntax time or during execution. I use browser DevTools, breakpoints, console output, and try/catch to find the failing line and inspect values.
>
> try/catch handles thrown errors in the relevant execution flow, including awaited Promise rejections. I do not silently ignore errors; I show useful messages and log details during development.

தமிழில் புரிந்து கொள்ள

Console, Network tab, breakpoint மூலம் error trace செய்யலாம். try/catch runtime errors handle செய்யும்.

SYNTAX

```
try { risky() } catch(error) { ... }
```

WORKING EXAMPLE

```
try {
  const value = JSON.parse("not-json");
  console.log(value);
} catch (error) {
  console.log("Invalid JSON");
}
```

Real-world use: API failures, malformed JSON, form problems.

Interview question: How do you debug a blank page?

06.02 Debouncing & Throttling

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Debouncing delays an action until calls stop for a specified period. It is useful for search input so an API request is not sent on every keystroke.
>
> Throttling limits how frequently a function can run during repeated events, such as scroll or resize. I choose debounce for the final user input and throttle for regular interval updates.

தமிழில் புரிந்து கொள்ள

Search type செய்யும் ஒவ்வொரு key-க்கும் API call வேண்டாம் என்றால் debounce. Scroll event frequent run ஆகாமல் throttle.

SYNTAX

```
clearTimeout(timer); timer=setTimeout(fn,delay);
```

WORKING EXAMPLE

```
let timer;
const search = text => {
  clearTimeout(timer);
  timer = setTimeout(() => {
    console.log("Search:", text);
  }, 400);
};
search("J"); search("JS");
```

Real-world use: Search suggestions and resize/scroll optimization.

Interview question: Debounce vs throttle?

06.03 Modules: import & export

SAY THIS IN YOUR INTERVIEW • ENGLISH

> JavaScript modules let me split code into reusable files. I export functions or values from one file and import them where needed.
>
> Named exports must be imported by their exported names, while a default export can be imported with a chosen local name. Modules make frontend code easier to maintain.

தமிழில் புரிந்து கொள்ள

Code-ஐ files-ஆ பிரித்து export/import பண்ணலாம். Large frontend projects-ல் reusable logic maintain செய்ய உதவும்.

SYNTAX

```
export const add = ...; import {add} from "./math.js";
```

WORKING EXAMPLE

```
// math.js
export const add = (a,b) => a+b;
// main.js
import { add } from "./math.js";
console.log(add(2,3));
// HTML: <script type="module" src="main.js"></script>
```

Real-world use: Reusable utilities, component-based applications.

Interview question: Named export vs default export?

06.04 JSON, HTTP Status & CORS Basics

SAY THIS IN YOUR INTERVIEW • ENGLISH

> JSON is a text format used to exchange structured data. Frontend applications often send and receive JSON over HTTP. HTTP status codes describe outcomes, such as 200 success, 201 created, 400 bad request, 401 unauthorized, 404 not found, and 500 server error.
>
> CORS is a browser security mechanism that controls whether a page can read responses from another origin. The server must send suitable CORS headers; it cannot be reliably fixed just by changing frontend JavaScript.

தமிழில் புரிந்து கொள்ள

JSON API data format. HTTP status response result சொல்லும். CORS issue backend/server permission-ஐப் பொறுத்தது.

SYNTAX

```
JSON.stringify(obj); JSON.parse(text);
```

WORKING EXAMPLE

```
const user = {name:"Arun"};
const text = JSON.stringify(user);
const parsed = JSON.parse(text);
console.log(parsed.name);
```

Real-world use: Understanding browser Network tab and API integration.

Interview question: What is CORS? What does 404 mean?

06.05 DOM Security & Safe Rendering

SAY THIS IN YOUR INTERVIEW • ENGLISH

> When displaying user-generated content, I prefer textContent because it treats the value as text rather than HTML. Directly inserting untrusted content into innerHTML can allow cross-site scripting.
>
> For simple trusted static templates, innerHTML is convenient, but real applications should escape or sanitize untrusted values or use safe DOM APIs. I never assume API data is automatically safe.

தமிழில் புரிந்து கொள்ள

User input-ஐ innerHTML-ல் direct போடுவது unsafe ஆகலாம். Plain text display-க்கு textContent safe.

SYNTAX

```
element.textContent = userInput;
```

WORKING EXAMPLE

```
// HTML: <p id="msg"></p>
const userInput = "<b>Hello</b>";
document.getElementById("msg").textContent = userInput;
// Displays literal text, not bold HTML
```

Real-world use: Comments, profile names, search results.

Interview question: What is XSS and how can frontend code reduce it?

06.06 Frontend Performance & Accessibility

SAY THIS IN YOUR INTERVIEW • ENGLISH

> Good frontend code minimizes unnecessary DOM work and network requests. I avoid repeatedly updating the DOM inside a large loop when I can build the output once.
>
> I also use semantic HTML, labels, meaningful button text, and keyboard-friendly controls. Accessibility improves usability and is an important sign of production-ready frontend work.

தமிழில் புரிந்து கொள்ள

Repeated DOM update avoid பண்ணலாம். Semantic HTML, label, keyboard access எல்லாம் professional frontend-க்கு important.

SYNTAX

```
const output = items.map(...).join("");
```

WORKING EXAMPLE

```
const items = ["HTML", "CSS", "JS"];
const output = items.map(x => `<li>${x}</li>`).join("");
// For trusted fixed strings only:
// document.getElementById("list").innerHTML = output;
```

Real-world use: Fast tables, usable forms, responsive applications.

Interview question: How do you optimize a slow list?

EX

Essential Interview Deep Dives

Expanded must-know explanations for frontend candidates. Study these in order: scope and hoisting → conversion and comparisons → DOM and forms → browser storage → asynchronous JS → performance and safety.

16 DETAILED INTERVIEW TOPICS

EX.01 Scope: Global, Function, Block & Lexical Scope

INTERVIEW SPEAKING ANSWER • ENGLISH

> Scope controls where variables can be accessed. A global variable is available to code in its environment, while a function-scoped variable stays inside its function. Variables declared with let and const are block-scoped, so an if or for block can keep temporary values private.
>
> Lexical scope means an inner function can access variables defined in its surrounding code. This is decided by where the function is written, not where it is called. Lexical scope is the foundation for understanding closures.

தமிழ் விளக்கம்

Scope என்பது variable-ஐ எந்த இடத்தில் access செய்யலாம் என்பதைக் குறிக்கும். Global variable வெளியிலும், function variable அந்த function-க்குள்ளும் இருக்கும். let, const block-க்குள் மட்டுமே accessible. Inner function outer variable-ஐ access செய்வது lexical scope.

SYNTAX

```
let globalValue = 10; function demo() { let local = 20; }
```

WORKING CODE AND EXPECTED OUTPUT

```
const company = "ABC";
const show = () => {
  let salary = 20000;
  if (true) {
    const bonus = 5000;
    console.log(salary + bonus); // 25000
  }
  console.log(company); // ABC
};
show();
// console.log(salary); // ReferenceError
```

Real-time frontend use: Form values, event handlers and counters without variable conflicts.

Interview question: Difference between global, function, block and lexical scope?

EX.02 Hoisting & Temporal Dead Zone

INTERVIEW SPEAKING ANSWER • ENGLISH

> Hoisting describes how JavaScript processes declarations before executing code. A var declaration is initialized as undefined before its assignment, so reading it early gives undefined. Function declarations can be called before their location in the source.
>
> let and const declarations are also processed before execution, but they cannot be accessed before initialization. That period is called the Temporal Dead Zone (TDZ). Reading them too early throws a ReferenceError.

தமிழ் விளக்கம்

Hoisting-னா declaration execute ஆகும் முன்பே JavaScript process பண்ணும். var-ஐ முன்னாடி read பண்ணா undefined. let/const-ஐ declaration line-க்கு முன்னாடி access பண்ணா ReferenceError வரும்; இதுதான் TDZ.

SYNTAX

```
console.log(a); var a = 10; // undefined
```

WORKING CODE AND EXPECTED OUTPUT

```
console.log(a); // undefined
var a = 10;
sayHi(); // Hi
function sayHi() { console.log("Hi"); }
// console.log(b); // ReferenceError
let b = 20;
```

Real-time frontend use: Avoiding initialization bugs and answering JS output questions.

Interview question: Why does var print undefined but let throw ReferenceError?

EX.03 Implicit & Explicit Type Conversion

INTERVIEW SPEAKING ANSWER • ENGLISH

> Type conversion changes a value from one data type to another. Implicit conversion happens automatically when JavaScript evaluates an expression, such as converting a string in numeric subtraction or converting a number into text during string concatenation.
>
> Explicit conversion is when I intentionally use Number(), String(), or Boolean() to convert values. In frontend forms, input.value is a string, so I explicitly convert it before numeric calculations to avoid incorrect results.

தமிழ் விளக்கம்

Implicit conversion JavaScript தானாக செய்யும்; explicit conversion நாமே Number(), String(), Boolean() வைத்து செய்வோம். Form input-ல value string-ஆ வரும். Salary, age, marks calculation செய்ய Number() use பண்ணணும்.

SYNTAX

```
Number('25'); String(100); Boolean(1);
```

WORKING CODE AND EXPECTED OUTPUT

```
console.log("5" + 2);   // "52"
console.log("5" - 2);   // 3
console.log(Number("25") + 5); // 30
console.log(String(50)); // "50"
console.log(Boolean("")); // false
console.log(Number("abc")); // NaN
```

Real-time frontend use: Converting age, marks, quantities and prices from form inputs.

Interview question: What is implicit vs explicit conversion? Why does '5' + 2 give '52'?

EX.04 Truthy, Falsy, ==, === & Nullish Values

INTERVIEW SPEAKING ANSWER • ENGLISH

> JavaScript uses truthy and falsy values in conditions. The falsy values include false, 0, -0, 0n, empty string, null, undefined and NaN. Other values, including empty arrays and empty objects, are truthy.
>
> The == operator can convert types before comparing, while === checks both type and value without coercion. The nullish coalescing operator ?? uses a fallback only for null or undefined; it does not replace valid values like 0 or an empty string.

தமிழ் விளக்கம்

if condition-ல் சில values false மாதிரி behave ஆகும்; அவை falsy. [] மற்றும் {} truthy. == type convert பண்ணி compare செய்யலாம்; === type-உம் value-உம் compare செய்யும். ?? null/undefined இருந்தால் மட்டும் fallback கொடுக்கும்.

SYNTAX

```
value === 0; const name = input ?? 'Guest';
```

WORKING CODE AND EXPECTED OUTPUT

```
console.log(0 == false);  // true
console.log(0 === false); // false
console.log(Boolean([])); // true
console.log(Boolean("")); // false
console.log(0 || 100); // 100
console.log(0 ?? 100); // 0
```

Real-time frontend use: Checking empty fields, API fallback values and exact ID comparisons.

Interview question: Difference between == and ===, and between || and ???

EX.05 DOM Selectors, Manipulation & Rendering

INTERVIEW SPEAKING ANSWER • ENGLISH

> The DOM is the browser's object representation of an HTML document. I can select elements using getElementById(), querySelector(), or querySelectorAll() and then update their text, attributes, styles or HTML in response to user actions.
>
> When displaying an array of objects, I can use forEach() to build table rows using template literals and assign the completed markup to innerHTML once. For production code, untrusted strings must be escaped or inserted safely to prevent HTML injection.

தமிழ் விளக்கம்

DOM என்பது HTML-ஐ JavaScript மூலம் access பண்ண உதவும் structure. ID, class, querySelector வைத்து element எடுத்து content மாற்றலாம். Array of objects-ஐ forEach + template literals வைத்து table-ல் display பண்ணலாம்.

SYNTAX

```
document.getElementById('output').textContent = 'Hi';
```

WORKING CODE AND EXPECTED OUTPUT

```
const users = [{id:1,name:"Arun"}, {id:2,name:"Priya"}];
let rows = "";
users.forEach((user) => {
  rows += `<tr><td>${user.id}</td><td>${user.name}</td></tr>`;
});
// HTML: <table><tbody id="body"></tbody></table>
document.getElementById("body").innerHTML = rows;
```

Real-time frontend use: Displaying students, employees and products dynamically.

Interview question: Difference between innerHTML, innerText and textContent?

EX.06 DOM Events, Event Object & Event Delegation

INTERVIEW SPEAKING ANSWER • ENGLISH

> An event is an action such as click, input, change or submit. addEventListener() registers a function that runs when the event occurs. The event object provides information such as event.target, which identifies the element that triggered the event.
>
> Event delegation means attaching one listener to a parent element and checking which child was clicked. It is useful for dynamically created rows and buttons because I do not need to attach a separate listener to every row.

தமிழ் விளக்கம்

Button click, typing, submit எல்லாம் events. addEventListener மூலம் function run பண்ணலாம். event.target எந்த element trigger பண்ணியது என்று காட்டும். Dynamic table-ல் parent-க்கு ஒரே listener வைத்து buttons handle பண்ணுவது event delegation.

SYNTAX

```
element.addEventListener('click', (event) => { ... });
```

WORKING CODE AND EXPECTED OUTPUT

```
// HTML: <div id="list"><button data-id="7">Delete</button></div>
document.getElementById("list").addEventListener("click", (e) => {
  if (e.target.matches("button[data-id]")) {
    console.log("Delete ID:", e.target.dataset.id);
  }
});
```

Real-time frontend use: Delete and edit buttons in dynamically generated tables.

Interview question: What is event bubbling and event delegation?

EX.07 DOM Form Handling, Validation & Array of Objects

INTERVIEW SPEAKING ANSWER • ENGLISH

> In form handling, I read input values, validate them, and respond to the submit event. Calling event.preventDefault() prevents the browser's default form submission and page navigation so that JavaScript can process the data.
>
> I create an object from the validated inputs, push it into an array, and re-render the table using forEach() and template literals. After saving, form.reset() clears the inputs. For real applications, I also handle duplicate IDs and safe HTML rendering.

தமிழ் விளக்கம்

Form submit-ல் preventDefault() வைத்து refresh தடுக்கிறோம். Input value எடுத்து validate செய்து object உருவாக்கி array-க்கு push செய்கிறோம். forEach + template literal மூலம் table refresh; form.reset() மூலம் fields clear.

SYNTAX

```
form.addEventListener('submit', (event) => { event.preventDefault(); });
```

WORKING CODE AND EXPECTED OUTPUT

```
// HTML: <form id="f"><input id="student" required>
// <button>Add</button></form><ul id="items"></ul>
const students = [];
document.getElementById("f").addEventListener("submit", (e) => {
  e.preventDefault();
  const name = document.getElementById("student").value.trim();
  if (!name) return;
  students.push({ id: students.length + 1, name });
  let html = "";
  students.forEach((s) => { html += `<li>${s.id}: ${s.name}</li>`; });
  document.getElementById("items").innerHTML = html;
  e.target.reset();
});
```

Real-time frontend use: Student registration, expense entries and employee records.

Interview question: Why preventDefault()? How do you add form data to an array and display it?

EX.08 localStorage, sessionStorage & JSON

INTERVIEW SPEAKING ANSWER • ENGLISH

> localStorage is browser storage that keeps string data across page refreshes and browser restarts for the same origin. sessionStorage also stores strings but is generally limited to the lifetime of the browser tab or page session.
>
> When saving arrays or objects, I use JSON.stringify() to convert them into a string. When reading, I use JSON.parse() to restore the original structure. Storage is useful for frontend demos and preferences, but it should not hold passwords or sensitive authentication secrets.

தமிழ் விளக்கம்

localStorage-ல் data refresh ஆனாலும் இருக்கும். sessionStorage tab session முடியும் வரை இருக்கும். Array/object-ஐ store செய்ய JSON.stringify(), read செய்ய JSON.parse() use பண்ணணும். Password போன்ற sensitive data store பண்ணக்கூடாது.

SYNTAX

```
localStorage.setItem('key', JSON.stringify(data));
```

WORKING CODE AND EXPECTED OUTPUT

```
const students = [{ id: 1, name: "Arun" }];
localStorage.setItem("students", JSON.stringify(students));
const saved = JSON.parse(localStorage.getItem("students") || "[]");
saved.forEach((s) => console.log(s.name));
localStorage.removeItem("students");
// localStorage.clear(); // clears all keys for this origin
```

Real-time frontend use: Remembering theme preference, draft forms and frontend-only demo records.

Interview question: Why do we need JSON.stringify and JSON.parse with localStorage?

EX.09 Closures & Remembered State

INTERVIEW SPEAKING ANSWER • ENGLISH

> A closure is created when an inner function retains access to variables in its outer lexical environment. This works even after the outer function has returned. It is useful when a function needs to remember values between calls.
>
> I store the returned inner function in a variable so I can call the same closure repeatedly. If I call the outer function again each time, it creates a new independent environment, so the remembered count starts over.

தமிழ் விளக்கம்

Outer function முடிந்தாலும் inner function outer variable-ஐ remember பண்ணும். Returned function-ஐ variable-ல் store செய்தால் அதே count maintain ஆகும். outer()() ஒவ்வொரு தடவையும் புதிய count create பண்ணும்.

SYNTAX

```
const result = outer(); result();
```

WORKING CODE AND EXPECTED OUTPUT

```
const counter = () => {
  let count = 0;
  return () => ++count;
};
const result = counter();
console.log(result()); // 1
console.log(result()); // 2
console.log(counter()()); // 1
```

Real-time frontend use: Private counters, reusable handlers and stateful utilities.

Interview question: Why does counter()() repeatedly print 1?

EX.10 Callbacks, Promises, Async/Await & Event Loop

INTERVIEW SPEAKING ANSWER • ENGLISH

> A callback is a function passed to another function for execution. A Promise represents the eventual result of an asynchronous operation and can be pending, fulfilled or rejected. then() handles fulfillment and catch() handles rejection.
>
> An async function returns a Promise. Inside it, await pauses that function until a Promise settles, without blocking the whole browser. Promise reactions run as microtasks, while setTimeout callbacks run as tasks, which matters in output prediction questions.

தமிழ் விளக்கம்

Callback என்பது function-ஐ இன்னொரு function-க்கு argument-ஆக pass செய்வது. Promise async result-ஐ handle செய்யும். async/await code-ஐ readable ஆக்கும். Promise.then callback microtask; setTimeout callback task.

SYNTAX

```
const result = await Promise.resolve(10);
```

WORKING CODE AND EXPECTED OUTPUT

```
console.log("A");
setTimeout(() => console.log("D"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("B");
// Output: A, B, C, D
const load = async () => {
  try { return await Promise.resolve("Done"); }
  catch (err) { console.error(err); }
};
```

Real-time frontend use: API loading, delayed operations and async interview output questions.

Interview question: Why does Promise.then execute before setTimeout(..., 0)?

EX.11 Fetch API, HTTP Methods & UI States

INTERVIEW SPEAKING ANSWER • ENGLISH

> The Fetch API sends HTTP requests and returns a Promise for the response. I use await fetch(url), check response.ok, and then parse JSON with await response.json(). A rejected network request is different from an HTTP error response such as 404.
>
> In a frontend application, I show loading while the request runs, render the data on success, and show a useful error or empty state if needed. GET reads resources, POST creates, PUT replaces, PATCH partially updates and DELETE requests removal.

தமிழ் விளக்கம்

Fetch API backend/API-லிருந்து data வாங்க உதவும். await fetch() response வரும் வரை wait பண்ணும்; response.ok check செய்யணும். Loading, success, empty, error UI states handle பண்ணுவது interview-க்கு முக்கியம்.

SYNTAX

```
const response = await fetch(url); const data = await response.json();
```

WORKING CODE AND EXPECTED OUTPUT

```
const getUsers = async () => {
  const box = document.getElementById("users");
  box.textContent = "Loading...";
  try {
    const response = await fetch(
      "https://jsonplaceholder.typicode.com/users"
    );
    if (!response.ok) throw new Error("HTTP " + response.status);
    const users = await response.json();
    box.textContent = users.length
      ? users.map((u) => u.name).join(", ")
      : "No users";
  } catch (error) {
    box.textContent = "Unable to load users";
  }
};
getUsers(); // HTML: <div id="users"></div>
```

Real-time frontend use: Users lists, product cards, search and dashboard API calls.

Interview question: Does fetch reject automatically for HTTP 404? How do you handle loading?

EX.12 Array Methods, Mutation & Immutability

INTERVIEW SPEAKING ANSWER • ENGLISH

> Arrays store ordered values. push(), pop(), shift() and splice() modify the original array, while map(), filter() and slice() return new arrays. forEach() runs a callback for each item and returns undefined.
>
> I choose the method based on the task: forEach for displaying, map for transforming, filter for selecting, find for one match, some/every for checks, and reduce for totals. I avoid changing the original array when predictable state updates are important.

தமிழ் விளக்கம்

Array-ல் data order-ஆக store ஆகும். push/splice original array-ஐ change செய்யும். map/filter new array return செய்யும். forEach display செய்ய, filter select செய்ய, find one item கண்டுபிடிக்க, reduce total calculate செய்ய பயன்படும்.

SYNTAX

```
arr.forEach(fn); arr.map(fn); arr.filter(fn);
```

WORKING CODE AND EXPECTED OUTPUT

```
const staff = [{name:"A",salary:20000},{name:"B",salary:40000}];
staff.forEach((e) => console.log(e.name));
const high = staff.filter((e) => e.salary > 30000);
const updated = staff.map((e) => ({...e, salary:e.salary+1000}));
const total = staff.reduce((sum,e) => sum+e.salary,0);
console.log(high.length, updated[0].salary, total);
// 1, 21000, 60000
```

Real-time frontend use: Employee filtering, salary calculations and UI list updates.

Interview question: Difference between map, forEach, filter, find and reduce?

EX.13 this, Arrow Functions & Function Binding

INTERVIEW SPEAKING ANSWER • ENGLISH

> The value of this depends on how a regular function is called. When invoked as an object method, this commonly refers to the object. Arrow functions do not create their own this; they capture it from their surrounding lexical scope.
>
> This difference matters for object methods and event handlers. I normally use regular method syntax when I need this to refer to an object, and arrow functions when I want a short callback or to retain the surrounding this.

தமிழ் விளக்கம்

Regular function-ல் this call செய்யும் விதத்தைப் பொறுத்து மாறும். Arrow function-க்கு தனி this கிடையாது; outer scope-இலிருந்து எடுக்கும். Object method-க்கு regular method syntax நல்லது.

SYNTAX

```
const obj = {name:'A', show(){ console.log(this.name); }};
```

WORKING CODE AND EXPECTED OUTPUT

```
const employee = {
  name: "Priya",
  show() { console.log(this.name); }
};
employee.show(); // Priya
const add = (a,b) => a+b;
console.log(add(2,3)); // 5
```

Real-time frontend use: Object methods, event listeners and callbacks.

Interview question: How is this different in regular and arrow functions?

EX.14 Shallow Copy, Deep Copy & Reference Types

INTERVIEW SPEAKING ANSWER • ENGLISH

> Primitive values such as numbers and strings are copied by value. Objects and arrays are reference values, so assigning one object variable to another does not make an independent copy.
>
> Spread syntax makes a shallow copy: the outer object is new, but nested objects remain shared. For supported data, structuredClone() creates a deep copy. This matters when updating UI data without accidentally changing the original object.

தமிழ் விளக்கம்

Object-ஐ வேற variable-க்கு assign செய்தால் இரண்டும் same reference. Spread {...obj} shallow copy; nested object இன்னும் share ஆகும். structuredClone() supported data-க்கு deep copy செய்யும்.

SYNTAX

```
const copy = {...original};
```

WORKING CODE AND EXPECTED OUTPUT

```
const original = {name:"Arun", address:{city:"Chennai"}};
const shallow = {...original};
shallow.address.city = "Madurai";
console.log(original.address.city); // Madurai
const deep = structuredClone(original);
deep.address.city = "Salem";
console.log(original.address.city); // Madurai
```

Real-time frontend use: Updating nested frontend state and preventing unexpected changes.

Interview question: What is the difference between shallow and deep copy?

EX.15 Debounce, Throttle & Search Performance

INTERVIEW SPEAKING ANSWER • ENGLISH

> Debouncing delays an action until calls have stopped for a chosen time. It is useful for search boxes because the API request runs only after the user pauses typing, instead of on every keystroke.
>
> Throttling limits how often an action can run within a time interval. It is useful for scroll and resize handlers. Both techniques reduce unnecessary work and improve the user experience.

தமிழ் விளக்கம்

Debounce: user typing நிறுத்திய பிறகு function run ஆகும். Throttle: ஒரு time interval-க்கு அதிகபட்சம் ஒரு தடவை மாதிரி limit பண்ணும். Search, resize, scroll performance-க்கு பயன்படும்.

SYNTAX

```
clearTimeout(timer); timer = setTimeout(callback, 400);
```

WORKING CODE AND EXPECTED OUTPUT

```
let timer;
const search = (value) => {
  clearTimeout(timer);
  timer = setTimeout(() => {
    console.log("Search API for:", value);
  }, 400);
};
search("j");
search("ja");
search("javascript"); // only final search runs
```

Real-time frontend use: Search-as-you-type interfaces and expensive UI updates.

Interview question: Difference between debounce and throttle?

EX.16 Error Handling, Optional Chaining & Defensive Code

INTERVIEW SPEAKING ANSWER • ENGLISH

> try/catch handles errors thrown during execution, including failures from awaited Promises. I use it to show useful feedback rather than allowing an API or parsing error to break the user interface.
>
> Optional chaining ?. safely accesses properties when an intermediate value may be null or undefined. Nullish coalescing ?? provides a fallback for those missing values. I also validate user input before calculations and submissions.

தமிழ் விளக்கம்

try/catch error வந்தால் handle செய்யும். API response-ல் சில field இல்லாமல் இருக்கலாம்; ?. வைத்து safely access செய்யலாம். ?? fallback value கொடுக்கும். Input validation எப்போதும் முக்கியம்.

SYNTAX

```
const city = user?.address?.city ?? 'Unknown';
```

WORKING CODE AND EXPECTED OUTPUT

```
const user = { name:"Arun" };
console.log(user?.address?.city ?? "Unknown");
try {
  const age = Number("abc");
  if (Number.isNaN(age)) throw new Error("Invalid age");
} catch (error) {
  console.log(error.message);
}
// Unknown; Invalid age
```

Real-time frontend use: Unreliable API responses, form validation and robust dashboards.

Interview question: Difference between optional chaining and nullish coalescing?

EX.17 JavaScript Modules, Strict Mode & Browser Security

INTERVIEW SPEAKING ANSWER • ENGLISH

> ES modules let me split JavaScript into separate files using export and import. This keeps functions reusable and helps organize a frontend application. Module scripts are strict by default.
>
> Strict mode prevents some unsafe JavaScript behavior, such as accidentally assigning to an undeclared variable. In frontend applications, I also avoid putting secret API keys in browser code and never trust unvalidated user input.

தமிழ் விளக்கம்

import/export மூலம் code-ஐ separate files-ல் reuse செய்யலாம். Module-கள் strict mode-ல் run ஆகும்.

Frontend JS-ல் secret key வைக்கக்கூடாது; user input safe-ஆ handle செய்யணும்.

SYNTAX

```
export const add = (a,b) => a+b; import {add} from './math.js';
```

WORKING CODE AND EXPECTED OUTPUT

```
// math.js
export const add = (a,b) => a+b;
// app.js
import { add } from "./math.js";
console.log(add(10,20)); // 30
// HTML: <script type="module" src="./app.js"></script>
```

Real-time frontend use: Organizing multi-page frontend applications and reusable utilities.

Interview question: Difference between normal scripts and ES modules?

LAB

Final Frontend Practical Assessment

Complete these without backend development. Use browser HTML, CSS and JavaScript only.

Task A — Form + DOM + localStorage
Build a student form with name, age and course. Validate fields; submit into an array of objects; show a table using forEach and template literals; persist and restore using JSON.stringify/parse; add edit and delete buttons. Explain preventDefault and event delegation.

Task B — Fetch + Promise + UI states
Fetch users from JSONPlaceholder, show Loading / Error / No Data, and display name/email/city. Add a debounced name search. Explain await, response.ok and JSON parsing.

Task C — Output prediction
Predict var vs let hoisting, == vs ===, '5' + 2 vs '5' - 2, closure counter, Promise.then vs setTimeout, shallow copy nested mutation.

Interview answer practice
For each of the 16 topics, speak two paragraphs in English, explain the concept in Tamil, write the syntax without looking, and give one frontend project example.

07

Hands-on Interview Practice

Start with loops and strings, then arrays and objects, then DOM and API tasks. Try without built-in shortcuts first; after that, explain the optimized approach.

01. Loops & numbers
Print numbers 1–100; replace multiples of 3 with Fizz, 5 with Buzz, and both with FizzBuzz.

02. Loops & numbers
Find factorial of 5 using a for loop; expected 120.

03. Loops & numbers
Check whether a number is prime without using array methods.

04. Loops & numbers
Reverse digits of 12345; expected 54321.

05. Strings
Reverse a string without using reverse(); input "hello", output "olleh".

06. Strings
Count vowels in "javascript"; show each vowel and the total.

07. Strings
Find the first non-repeating character in "swiss"; expected "w".

08. Strings
Count character frequency in "banana"; expected b:1, a:3, n:2.

09. Arrays
Find the second largest distinct number in [10,45,20,45,30]; expected 30.

10. Arrays
Remove duplicates from [1,2,2,3,4,4,5] without Set.

11. Arrays
Find the largest and smallest numbers without sort().

12. Arrays
Find the missing number in [1,2,4,5]; expected 3.

13. Arrays
Move all zeros in [0,1,0,3,12] to the end while keeping other numbers in order.

14. Objects
Count employees earning more than 30000 and calculate their total salary.

15. Objects
Update the salary of employee id 2 by 5000 without changing other employees.

16. Objects
Group students by course using an array of objects.

17. DOM
Build a student form with name, age, course; on submit push a new object into an array.

18. DOM
Render student records in a table using forEach and template literals.

19. DOM
Add edit and delete buttons to each row using a stable unique id.

20. DOM
Add search input that filters a displayed employee table by name.

21. DOM
Validate a registration form and display inline errors.

22. Async
Create a closure counter that increments by 10 every call.

23. Async
Write a callback-based calculator with add, subtract, and multiply operations.

24. Async
Create a Promise that resolves for marks >=35 and rejects otherwise.

25. Async
Predict output order of console.log, Promise.then, and setTimeout.

26. API
Fetch users from JSONPlaceholder and display name, email, city in a table.

27. API
Show Loading, Error, and No Data states during API requests.

28. API
Create a POST request using JSONPlaceholder and show the returned result.

29. API
Build a user search input with 400ms debounce.

30. API
Handle a 404 response using response.ok and try/catch.

08

Interview Output Questions

First predict the output, then run the code. Explain why each output appears.

01. Equality

```
console.log(0 == false);
console.log(0 === false);
```

Answer: true, false

Why: Loose equality coerces types; strict equality does not.

02. Array reference

```
const a = [1];
const b = [1];
console.log(a === b);
```

Answer: false

Why: The arrays are different objects.

03. Scope

```
let x = 5;
{ let x = 10; }
console.log(x);
```

Answer: 5

Why: The inner x is block-scoped.

04. Closure

```
const make = () => { let x = 0; return () => ++x; };
const f = make();
console.log(f(), f());
```

Answer: 1 2

Why: The same closure remembers x.

05. Event loop

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
```

Answer: A D C B

Why: Synchronous code, then microtasks, then timer task.

06. Coercion

```
console.log("10" + 5);
console.log("10" - 5);
```

Answer: 105, 5

Why: Plus concatenates with a string; minus coerces to number.

07. Nullish

```
console.log(0 || 99);
console.log(0 ?? 99);
```

Answer: 99, 0

Why: ?? only falls back on null/undefined.

08. Map

```
const a = [1,2,3];
console.log(a.map(x => x * 2));
```

Answer: [2,4,6]

Why: map returns a new array.

09. Hoisting

```
console.log(a);
var a = 7;
```

Answer: undefined

Why: var declaration is hoisted with undefined.

10. Function return

```
const test = () => { 5 + 5; };
console.log(test());
```

Answer: undefined

Why: A block-bodied arrow needs explicit return.

09

4-Week Revision Roadmap

Adapt the schedule to your class pace. Each week includes speaking, coding, and revision.

Week	Topics	Must complete
1	Variables, types, operators, conditions, loops, functions, strings	10 logic questions; explain == vs ===, var/let/const
2	Arrays, objects, methods, destructuring, scope, closures, callbacks	10 array/object tasks; explain closure with counter
3	DOM, events, forms, table rendering, localStorage	One working student CRUD mini-project
4	Promises, async/await, Fetch API, errors, HTTP, revision	One users API mini-project; 2 mock interviews

Three frontend-only portfolio projects

1. Student Management — Add, edit, delete, search and validate students. Start with an array of objects, then optionally persist in localStorage.

2. Product Explorer — Fetch products from a public demo API; show loading/error states, cards, category filters and search.

3. Employee Dashboard — Display employees in a table, calculate total salary, filter by department, and sort by salary.

Interview answer formula

Definition → How it works → Small code example → Real use. For example: “A closure is an inner function that remembers variables from its outer scope. I can return that function and store it in a variable, so repeated calls preserve the counter. I would use this for a private counter or reusable handler.”

Final checklist

- ☐ Explain 50+ core concepts in simple English.
- ☐ Predict common output questions correctly.
- ☐ Write loops and array/object logic without relying on memorized answers.
- ☐ Build and validate a DOM form.
- ☐ Render an array of objects with forEach and template literals.
- ☐ Explain closures, callbacks, Promises and async/await.
- ☐ Fetch API data and handle HTTP errors, loading and empty states.
- ☐ Explain one frontend project end-to-end.

Reference: ECMAScript language concepts, browser DOM, Fetch API and HTTP fundamentals. Example APIs are intended for practice; network access is required to run them. This is a frontend-focused preparation guide, not a backend course.